UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

Visión geométrica · Procesamiento de imágenes

Identificador de figuras geométricas con OpenCV y grafo de similitud

Reporte técnico del proyecto «examen.html»

Alumno: Eduardo Parrilla Toledano

Grupo: 172

Fecha: 28 de octubre de 2026

1. Descripción general del código

El archivo **examen.html** es una aplicación web de una sola página que identifica figuras geométricas dibujadas por el usuario. El usuario puede trazar una silueta con el ratón o el dedo sobre un *canvas* de 640×500 píxeles, cargar una imagen o elegir una de cinco figuras de referencia (círculo, cuadrado, triángulo, trapecio y estrella). Al pulsar «Identificar figura», el sistema extrae cuatro rasgos geométricos con **OpenCV.js** (la versión de OpenCV para JavaScript, cargada desde docs.opencv.org), los compara con un prototipo por clase y muestra la figura predicha, el porcentaje de confianza y los valores de los rasgos.

Además de la clasificación, el programa dibuja un **grafo de similitud** en SVG: cada clase es un nodo y cada par de clases se une con una arista cuyo peso indica qué tan parecidos son sus vectores de rasgos. La clase **null** representa una entrada vacía, incompleta o que no coincide lo suficiente con ninguna figura conocida.

Estructura del código

- **HTML/CSS:** interfaz con dos paneles: entrada de la figura (canvas, botones, resultado y rasgos) y grafo de similitud (SVG).
- **Entrada:** eventos `pointerdown/pointermove/pointerup` para dibujar, `makeShape()` para generar figuras de ejemplo y carga de imágenes con `FileReader/URL.createObjectURL`.
- **Visión por computadora:** `getFeatures()` aplica el flujo de OpenCV y devuelve el vector de rasgos.
- **Clasificación:** `distance()` y `classify()` comparan el vector contra los prototipos.
- **Grafo:** `drawGraph()` construye nodos y aristas ponderadas con elementos SVG.

2. Metodología OpenCV

El análisis se basa en **contornos**. El lienzo se convierte en una matriz de imagen y se procesa con la siguiente secuencia de funciones de OpenCV:

Paso	Función OpenCV	Propósito
1	cv.imread(canvas)	Lee el contenido del canvas como matriz RGBA.
2	cv.cvtColor(…, COLOR_RGBA2GRAY)	Convierte a escala de grises.
3	cv.threshold(…, THRESH_BINARY_INV + THRESH_OTSU)	Binariza con el umbral óptimo de Otsu; la figura queda en blanco (255) sobre fondo negro. Si 3 de 4 esquinas resultan blancas, se invierte la imagen con cv.bitwise_not.
4	cv.findContours(…, RETR_EXTERNAL, CHAIN_APPROX_SIMPLE)	Obtiene los contornos externos; se elige el de mayor área (mínimo 300 px²).
5	cv.contourArea / cv.arcLength / cv.boundingRect	Calcula el área, el perímetro y el rectángulo delimitador del contorno.
6	cv.approxPolyDP(…, ε = máx(2, 0.02·P), true)	Aproxima el contorno por un polígono; el número de filas es el número de vértices.
7	cv.flip + cv.absdiff + cv.mean	Refleja la figura horizontalmente y mide la diferencia para obtener la simetría.

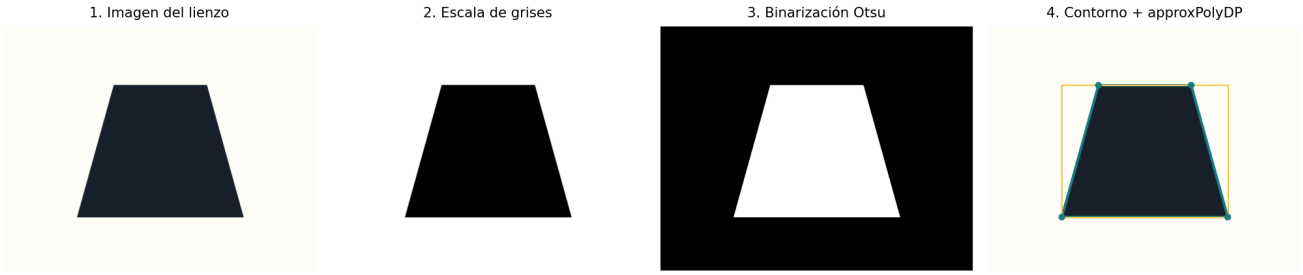


Figura 1. Flujo de procesamiento aplicado a un trapecio: imagen original, escala de grises, binarización de Otsu y contorno con su aproximación poligonal (vértices en verde azulado, rectángulo delimitador en amarillo).

3. Figuras de entrada y resultados

La Figura 2 muestra las cinco figuras de referencia que genera el botón de ejemplos (arriba) y el contorno detectado por OpenCV con su aproximación poligonal (abajo). Las imágenes se reprodujeron con el mismo algoritmo y las mismas coordenadas del código, usando OpenCV en Python.

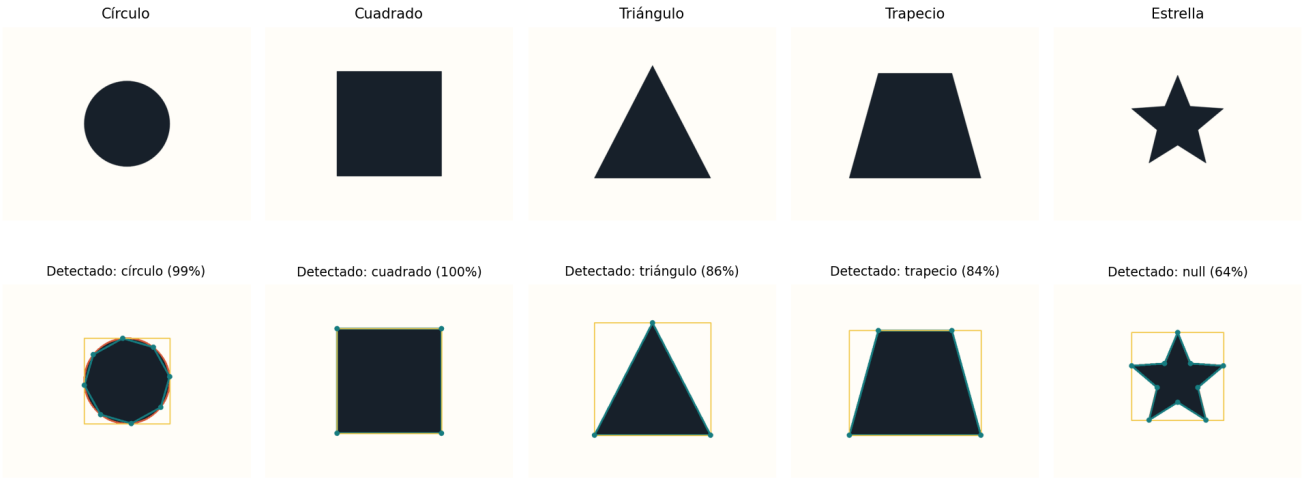


Figura 2. Figuras de referencia y resultado de la detección de contornos, con la clase predicha y su confianza.

Figura	Área ocupada	Ancho/alto	Vértices	Simetría	Predicción	Confianza
Círculo	0.78	1.00	0	100%	círculo	99%
Cuadrado	0.99	1.00	4	100%	cuadrado	100%
Triángulo	0.50	1.03	3	100%	triángulo	86%
Trapecio	0.78	1.26	4	100%	trapecio	84%
Estrella	0.38	1.05	10	100%	null	64%

Tabla 1. Rasgos extraídos y clase predicha para cada figura de referencia.

4. Grafo propuesto

El grafo es **completo, no dirigido y ponderado**: tiene 6 nodos (las cinco figuras y la clase *null*) y 15 aristas. Cada nodo está asociado al vector prototipo de su clase y el peso de cada arista expresa la similitud entre dos prototipos; el grosor de la línea es proporcional a ese peso.

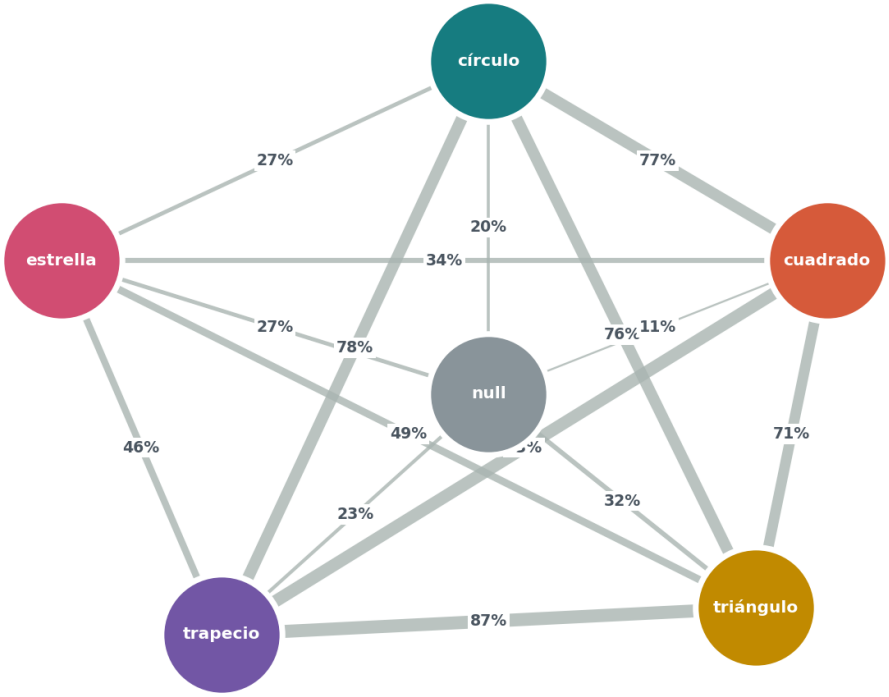


Figura 3. Grafo de similitud entre clases. Grosor de la arista = similitud; el porcentaje es el peso de la arista.

Arista	Peso	Arista	Peso	Arista	Peso
círculo – cuadrado	77%	cuadrado – triángulo	71%	triángulo – estrella	49%
círculo – triángulo	76%	cuadrado – trapecio	83%	triángulo – null	32%
círculo – trapecio	78%	cuadrado – estrella	34%	trapecio – estrella	46%
círculo – estrella	27%	cuadrado – null	11%	trapecio – null	23%
círculo – null	20%	triángulo – trapecio	87%	estrella – null	27%

Tabla 2. Pesos de las 15 aristas del grafo.

Los pares más parecidos son **triángulo–trapecio (87%)** y **cuadrado–trapecio (83%)**, lo que resulta coherente: son polígonos de 3 a 4 vértices con alta simetría. La estrella, con 10 vértices y baja ocupación del área, es la clase más alejada de las demás, y *null* tiene los pesos más bajos por ser el vector nulo.

5. Formulación matemática

Vector de rasgos. Cada figura se representa con cuatro números, donde A es el área del contorno, (w, h) el ancho y alto de su rectángulo delimitador, P el perímetro, v el número de vértices y s la simetría:

$$x = [\, x_1, x_2, x_3, x_4 \,] = [\, A/(w \cdot h),\ w/h,\ v,\ s \,]$$

Circularidad y vértices. Para distinguir círculos de polígonos se usa la circularidad, y el número de vértices se obtiene de la aproximación poligonal con tolerancia $\varepsilon = \text{máx}(2, 0.02 \cdot P)$:

$$C = 4\pi A / P^2 \qquad v = 0 \text{ si } C > 0.82; \text{ en otro caso } v = \text{mín}(12, \text{máx}(3, n_{approx}))$$

Simetría. Sea R la región de la figura binarizada y R′ su reflejo horizontal. La simetría es el porcentaje de píxeles que coinciden:

$$s = 100 - (\ \text{media}|R - R'| \,/\, 255\ ) \cdot 100$$

Distancia normalizada. Para comparar el vector x con un prototipo p se escalan los rasgos con $\sigma = [1, 1, 10, 100]$ para que todos pesen de forma comparable:

$$d(x, p) = \sqrt{(\ (1/4) \cdot \Sigma_{i=1..4}\ (\ (x_i - p_i) / \sigma_i\ )^2\ )}$$

Clasificación y confianza. Se elige la clase c* con menor distancia $d^* = \text{mín}\ d(x, p_c)$. Si $d^* > 0.35$ la salida es *null*; la confianza es:

$$\text{confianza} = \text{máx}(\ 0, \text{redondear}(\ (1 - d^*) \cdot 100\ )\ )\ \%$$

Peso de las aristas del grafo. Entre dos clases i y j:

$$w_{ij} = \text{máx}(\ 0, \text{redondear}(\ 100 \cdot (1 - d(p_i, p_j))\ )\ )\ \%$$

Prototipos usados por el código

Clase	Área ocupada	Ancho/alto	Vértices	Simetría
Círculo	0.785	1	0	98
Cuadrado	1	1	4	100
Triángulo	0.5	1	3	73
Trapecio	0.72	1	4	82
Estrella	0.38	1	10	0
Null	0	0	0	0

Tabla 3. Vectores prototipo definidos en el objeto `prototypes`.

6. Análisis de resultados

Cuatro de las cinco figuras de referencia se clasifican correctamente: círculo (99%), cuadrado (100%), triángulo (86%) y trapecio (84%). En el círculo, la circularidad medida es 0.90 (> 0.82), por lo que se asigna v = 0 y coincide con su prototipo.

Caso de la estrella. Al reproducir el algoritmo, la estrella se clasifica como *null*. Los rasgos medidos son correctos (área 0.38, 10 vértices), pero la simetría real es ≈100% y el prototipo de la estrella tiene simetría 0. Esto eleva la distancia a su propia clase a 0.50, mayor que la distancia al trapecio (0.36), y como la mejor distancia supera el umbral de 0.35 el resultado es *null*. Es muy probable que el valor 0 del prototipo sea un error de captura: una estrella regular de cinco puntas es simétrica respecto al eje vertical, por lo que el valor esperado sería cercano a 100. Con ese ajuste, la estrella quedaría clasificada con confianza muy alta. Conviene verificarlo ejecutando el HTML en el navegador.

7. Conclusiones

- El proyecto demuestra un clasificador geométrico **basado en reglas y prototipos**, sin entrenamiento, apoyado en OpenCV.js: umbralización de Otsu, extracción de contornos, aproximación poligonal y medición de simetría.
- Los cuatro rasgos (área ocupada, relación ancho/alto, vértices y simetría) son suficientes para separar círculo, cuadrado, triángulo y trapecio, y son invariantes a la posición de la figura en el lienzo.
- La distancia normalizada y el umbral de 0.35 permiten rechazar entradas desconocidas mediante la clase *null*, lo que evita falsas clasificaciones con lienzos vacíos o trazos ambiguos.
- El **grafo ponderado** hace visible la relación entre clases: las figuras con pesos altos (triángulo–trapecio, cuadrado–trapecio) son las más propensas a confundirse, mientras que la estrella queda aislada.
- **Limitaciones:** la simetría solo se mide respecto al eje vertical, la figura debe estar completa y cerrada, y la clasificación depende de que los prototipos estén bien calibrados, como lo muestra el caso de la estrella.
- **Mejoras propuestas:** corregir el prototipo de simetría de la estrella, calibrar los prototipos con muchos dibujos reales, añadir rasgos invariantes a la rotación (por ejemplo, momentos de Hu con cv.HuMoments) y usar un clasificador como k-NN para aprender los prototipos de los datos.